

Figure 16.3.1. Example of a Markov model. Transitions occur between states along the directed edges shown. Each outgoing edge is labeled by its probability. The sum of the probabilities on the outgoing edges from each state is 1. This model is called “Teen Life.”

16.3 Markov Models and Hidden Markov Modeling

Trellises, like those in §16.2, are directed graphs without any loops, so a path that begins at the leftmost node inevitably ends, after a finite number of stages, at the rightmost node. The more general *Markov model* lives on a graph that can have loops (as in Figure 16.2.1), so some paths can continue indefinitely. Indeed, as a convention, one can add a self-loop (a directed edge connecting a state to itself) to any state that would otherwise have “no way out.” Then, all paths can be continued indefinitely, even those whose fate is to remain stuck in a single state.

To turn such a directed graph into a Markov model (also known as a *Markov chain* or *first-order Markov process*), we simply label all of its edges with *transition probabilities*, such that the sum of probabilities over the outgoing edges from each node is 1. Figure 16.3.1 shows an example, a Markov model with five states, that we call “Teen Life.”

A single realization of a Markov model is a random path that moves from state to state according to the model’s probabilities. These are conveniently organized into a *transition matrix* $\mathbf{A}$ whose element A_{ij} is the probability associated with an $i \rightarrow j$ transition, that is, the probability of moving to state j , given state i as the starting point. A valid transition matrix satisfies

$$0 \leq A_{ij} \leq 1 \quad \text{and} \quad \sum_j A_{ij} = 1 \quad (16.3.1)$$

The transition matrix for Teen Life (Figure 16.3.1) is

$$\mathbf{A} = \begin{pmatrix} 0 & 0.7 & 0.1 & 0 & 0.2 \\ 0.2 & 0.4 & 0 & 0.2 & 0.2 \\ 0 & 1.0 & 0 & 0 & 0 \\ 0 & 0.3 & 0 & 0.7 & 0 \\ 0.1 & 0.1 & 0 & 0 & 0.8 \end{pmatrix} \quad (16.3.2)$$

where the states are numbered in the order (Eat, Hang, Study, Talk, and Sleep).

A routine for generating a realization of a Markov model from its $M \times M$ transition matrix, using the `Ran` structure of §7.1 to get random numbers, is straightforward.

```
void markovgen(const MatDoub_I &atrans, VecInt_0 &out, Int istsart=0,
               Int seed=1) {
```

markovgen.h

Generate a realization of an M -state Markov model, given its $M \times M$ transition matrix `atrans`. The vector `out` is filled with integers in the range $0 \dots M-1$. The starting state is the optional argument `istsart` (defaults to 0). `seed` is an optional argument that sets the seed of the random number generator.

```
    Int i, ilo, ihi, ii, j, m = atrans.nrows(), n = out.size();
    MatDoub cum(atrans);           Temporary matrix to hold cumulative probabilities.
    Doub r;
    Ran ran(seed);                 Use the random number generator Ran.
    if (m != atrans.ncols()) throw("transition matrix must be square");
    for (i=0; i<m; i++) {          Fill cum and die if clearly not a transition matrix.
        for (j=1; j<m; j++) cum[i][j] += cum[i][j-1];
        if (abs(cum[i][m-1]-1.) > 0.01)
            throw("transition matrix rows must sum to 1");
    }
    j = istsart;                   The current state is kept in j.
    out[0] = j;
    for (ii=1; ii<n; ii++) {       Main loop.
        r = ran.doub()/cum[j][m-1]; Slightly-off normalization gets corrected here.
        ilo = 0;
        ihi = m;
        while (ihi-ilo > 1) {       Use bisection to find location among the cumulative probabilities.
            i = (ihi+ilo) >> 1;
            if (r>cum[j][i-1]) ilo = i;
            else ihi = i;
        }
        out[ii] = j = ilo;         Set new current state.
    }
}
```

What makes the transition matrix a matrix, and not just a table of probabilities, is its connection to *ensembles* of realizations of the corresponding Markov model. An ensemble can be characterized by the components of a *population vector* $\mathbf{s}_t$ whose components give the number of models in each state at time t . (Here and below, we use t as an integer, discrete time variable. On a trellis it would be called a *stage* instead of a *time*.) For Teen Life, we can give names to the components of $\mathbf{s}_t$ corresponding to the states: (E, H, S, T, Z) .

If all the models in the ensemble are evolved by one timestep (one transition), then the population vector $\mathbf{s}_t$ turns into $\mathbf{s}_{t+1}$ by the matrix multiplication

$$\mathbf{s}_{t+1} = \mathbf{A}^T \mathbf{s}_t \quad (16.3.3)$$

The transpose operation is needed only because two common conventions are at odds: The time order $i \rightarrow j$ for A_{ij} versus the left matrix multiplication of a column vector (whose implicit time ordering is “from” the second index “to” the first). Given a matrix, you can easily tell whether it is intended to be an $\mathbf{A}$ or an $\mathbf{A}^T$, by whether, respectively, its rows or columns sum to unit probability.

Note that we can evolve more than one step at a time, by precomputing powers of $\mathbf{A}^T$. So, $\mathbf{s}_{t+n} = (\mathbf{A}^T)^n \mathbf{s}_t$, for example.

Every Markov model has at least one equilibrium distribution of states that remains unaffected when multiplied by $\mathbf{A}^T$. To prove this, we write

$$\mathbf{A}^T \mathbf{s}_e = \mathbf{s}_e \quad \Longleftrightarrow \quad (\mathbf{A}^T - \mathbf{1}) \mathbf{s}_e = \mathbf{0} \quad (16.3.4)$$

where $\mathbf{s}_e$ is the sought-after equilibrium state. Equation (16.3.4) can hold if and only if $\mathbf{A}^T - \mathbf{1}$ is a singular matrix. Since the columns of $\mathbf{A}^T$ all sum to 1, the columns

of $\mathbf{A}^T - \mathbf{1}$ all sum to zero, and hence are linearly dependent, q.e.d. Equivalently, we have proved that $\mathbf{A}^T$ has at least one eigenvalue equal to 1. The corresponding eigenvector is an equilibrium distribution of states. If there is only one eigenvalue equal to 1, the equilibrium is unique. For the Teen Life model, there is one eigenvalue of 1, and the corresponding eigenvector (normalized so that its components sum to unity) is approximately (0.099, 0.297, 0.001, 0.198, 0.395). (This teenager spends about 39.5% of his/her time sleeping, 19.8% talking on the phone, 0.1% studying, and so forth.)

Do almost all starting distributions converge to a unique equilibrium, in which case the model is said to be *ergodic*? Not necessarily. Two things can go wrong. First, if there is more than one eigenvalue equal to 1, a model will converge to some different linear combination of the corresponding eigenvectors for different starting distributions. Such models are said to fail the test of *irreducibility*. Second, the model may have a periodic limit cycle, so that, for most starting distributions, it doesn't converge at all. Such models are said to fail the test of *aperiodicity*. The theorem (and vocabulary test) is: Irreducibility and aperiodicity imply ergodicity.

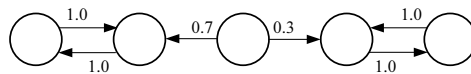
One way to diagnose these conditions is to perform successive squarings of the matrix $\mathbf{A}^T$ to take it to a very high power, say 2^{32} . This requires $O(32 M^3)$ operations for a model with M states. (While there are more sophisticated methods, none scale better than M^3 .) If all M columns in the result are converging to identical vectors, then there is just one eigenvalue (unity), and all starting distributions will converge to its eigenvector (which is in fact the repeated column vector). The model is then ergodic.

Otherwise, locate any rows in the power matrix that are zero, and cross out those rows and their corresponding columns. (These are states that become permanently unpopulated as the model evolves.) Then, check to see if the remaining columns are all eigenvectors with unit eigenvalue. You can do the test by multiplying each such column by the original $\mathbf{A}^T$. If all columns pass the test, then there are multiple equilibria, but all starting distributions will converge to some combination of them. If any column fails the test, then the model has a periodic limit cycle. There are still equilibria, given by the eigenvectors of unit eigenvalue, but a starting state must be very special to evolve to one. Indeed, such states are a set of measure zero, and we can say that the equilibria are *unstable*.

A simple example with multiple equilibria and periodic limit cycles is the transition matrix

$$\mathbf{A}' = \begin{pmatrix} 0 & 1.0 & 0 & 0 & 0 \\ 1.0 & 0 & 0 & 0 & 0 \\ 0 & 0.7 & 0 & 0.3 & 0 \\ 0 & 0 & 0 & 0 & 1.0 \\ 0 & 0 & 0 & 1.0 & 0 \end{pmatrix} \quad (16.3.5)$$

corresponding to the graph



$\mathbf{A}'^T$ has two eigenvectors with unit eigenvalue (you can guess them from the graph): (0.5, 0.5, 0, 0, 0) and (0, 0, 0, 0.5, 0.5). However, $\mathbf{A}'^T$ to the power 2^{32} does not have

these as any of its columns, but is rather

$$(\mathbf{A}'^T)^{2^{32}} = \begin{pmatrix} 1.0 & 0 & 0.7 & 0 & 0 \\ 0 & 1.0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1.0 & 0 \\ 0 & 0 & 0.3 & 0 & 1.0 \end{pmatrix} \quad (16.3.6)$$

thus showing that the model has only unstable equilibria. (The little identity matrix blocks are merely artifacts of the limit cycles having period 2, while 2^{32} is even. In general, you will get some other pattern.)

Successive squaring is a poor way to get the equilibrium distribution for a model that is known (or guessed) to have a single stable equilibrium. A better way, since we already know the eigenvalue, is inverse iteration. Just solve the equation

$$(\mathbf{A}^T - \mathbf{1}) \mathbf{s}_e = \mathbf{b} \quad (16.3.7)$$

by *LU* decomposition (§2.3), with the right-hand vector $\mathbf{b} = (1, 1, \dots, 1)$. If your solver complains about the zero pivot instead of substituting a small value for it (which is what we want for this application), then use the matrix $(\mathbf{A}^T - 0.999999 \times \mathbf{1})$ instead. In either case, you will want to renormalize $\mathbf{s}_e$, to make, e.g., its components have unit sum.

You can test for multiple equilibria by perturbing the right-hand side vector and seeing if you get the same $\mathbf{s}_e$. If you do have multiple equilibria, it is probably time to turn to the methods of Chapter 11 and calculate $\mathbf{A}^T$'s eigenvalues and eigenvectors directly.

That is all (in fact, more) than we want to tell about Markov models in general. We turn now to the real business at hand, which is to estimate statistically the state of a “hidden” Markov model, given only partial or imperfect information.

16.3.1 Hidden Markov Models

In a *hidden Markov model* (HMM), we don't get to observe the state of the model directly. Rather, whenever it is in any state i (one of M states), it emits a *symbol* k , chosen probabilistically from a set of K symbols. The probability of emitting symbol number k from state number i is denoted by

$$b_i(k) \equiv P(\text{symbol } k \mid \text{state } i) \quad (0 \leq i < M, \quad 0 \leq k < K) \quad (16.3.8)$$

with the normalization condition

$$\sum_{k=0}^{K-1} b_i(k) = 1 \quad (0 \leq i < M) \quad (16.3.9)$$

Thus, when the model evolves through N timesteps, the *hidden states* are a vector of integers,

$$\mathbf{s} = \{s_t\} = (s_0, s_1, \dots, s_{N-1}) \quad (16.3.10)$$

each in the range $0 \leq s_i < M$, while the *observations* or *data* are a vector of integers,

$$\mathbf{y} = \{y_t\} = (y_0, y_1, \dots, y_{N-1}) \quad (16.3.11)$$

each in the range $0 \leq y_i < K$.

For the Teen Life example, here is a table of symbols and their probabilities of being emitted from each state, in response to the repeated parental query, “What are you doing?”:

			$i = 0$	1	2	3	4
k	symbol	meaning	Eat	Hang	Study	Talk	Sleep
0	o	[silence]	0.2	0.2	0	0.3	0.5
1	s	“I’m studying!”	0	0	1.0	0.2	0
2	b	“I’m busy!”	0	0.6	0	0.4	0
3	g	[grunt]	0.8	0.2	0	0.1	0
4	z	[snore]	0	0	0	0	0.5

The key point is that the emitted symbols give only incomplete, or garbled, state information (e.g., the claim of studying when actually talking on the phone). A state can emit more than one possible symbol, and a symbol can be emitted by more than one possible state. Nevertheless, our goal is to make the best statistical reconstruction of the vector $\mathbf{s}$ from $\mathbf{y}$.

More specifically, at each time t we want to estimate

$$P_t(i) \equiv P(s_t = i \mid \mathbf{y}) \quad (16.3.12)$$

the probability that the actual state of the system is i at time t , given all the data. (If the word “probability” in this context bothers you, you may think of it as a likelihood.)

Let $\alpha_t(i)$ be defined for $t = 0 \dots N - 1$ and $i = 0 \dots M - 1$ as the probability of the observed data up to time t (that is, $y_0 \dots y_t$), *given* that we are in state i at time t . Since we are not specifying any of the previous states, we must sum over all possible paths that get to state i at time t . Thus,

$$\begin{aligned} \alpha_0(i) &= b_i(y_0) \\ \alpha_t(i) &= \sum_{i_0, i_1, \dots, i_{t-1}} b_{i_0}(y_0) A_{i_0 i_1} b_{i_1}(y_1) \dots A_{i_{t-1} i} b_i(y_t) \quad (1 \leq t < N) \end{aligned} \quad (16.3.13)$$

In other words, each transition contributes to the product both a transition probability and a symbol probability, and we sum over all possible combinations of previous states, that is, all possible values of $i_0, i_1, \dots, i_{t-1}$, each in the range $0 \dots M - 1$.

Since $\alpha_t(i)$ is the probability of data given state, it can also be interpreted as the likelihood of the state, given the data; or, if we are Bayesians (and we are!), as the unnormalized posterior probability of being in state i , which can be normalized simply by dividing by $\sum_i \alpha_t(i)$. However, equation (16.3.13) seems useless for a big and a little reason. Big: It has exponentially many terms to evaluate. Little: it uses only part of the data (the data earlier in time) to estimate the state i at time t . It is what is called a *forward estimate*.

Amazingly, both problems are easy to fix. It is not hard to see that the $\alpha_t(i)$ ’s

satisfy a recurrence relation that advances them all one step in t :

$$\alpha_{t+1}(j) = \sum_{i=0}^{M-1} \alpha_t(i) A_{ij} b_j(y_{t+1}) \quad (0 \leq j < M) \quad \text{for } t = 0, \dots, N-2 \quad (16.3.14)$$

One step of this recurrence takes only $O(M^2)$ operations, so the whole table of $\alpha_t(i)$'s can be computed in $O(NM^2)$.

To fix the second problem, we come at the issue from “the other end of time.” Let $\beta_t(i)$ be defined for $t = N-1 \dots 0$ and $i = 0 \dots M-1$ as the probability of the *future* observed data $(y_{t+1}, y_{t+2}, \dots, y_{N-1})$ again *given* that we are in state i at time t . Analogously to the α 's, we have

$$\beta_t(i) = \sum_{i_{t+1}, \dots, i_{N-1}} A_{ii_{t+1}} b_{i_{t+1}}(y_{t+1}) \dots A_{i_{N-2}i_{N-1}} b_{i_{N-1}}(y_{N-1}) \quad (16.3.15)$$

with the special case $\beta_{N-1}(i) = 1$. (Because there are no data to the future of $t = N-1$, those data's probability is by definition 1.) In the formula for the $\beta_t(i)$'s there is a factor in the product for each future transition probability and each future symbol probability (fixing the symbols to be the actual y 's of course). Just as for the α 's, the β 's can be interpreted as likelihoods, or unnormalized posterior probabilities. And, wonderfully, equation (16.3.15) can be solved by a *backward* recurrence,

$$\beta_{t-1}(i) = \sum_{j=0}^{M-1} A_{ij} b_j(y_t) \beta_t(j) \quad (0 \leq i < M) \quad \text{for } t = N-1, \dots, 1 \quad (16.3.16)$$

Calculating all the β 's for $t = N-1, N-2, \dots, 0$ is called a *backward estimate*.

Now here is the big payoff: From the definitions of the α 's and β 's, the product $\alpha_t(i)\beta_t(i)$ is the unnormalized posterior probability of state i at time t given *all* the data. If we normalize it by dividing by

$$\mathcal{L}_t = \sum_{i=0}^{M-1} \alpha_t(i) \beta_t(i) \quad (16.3.17)$$

we get the desired estimate of the probability of each separate state at each separate time,

$$P_t(i) = \frac{\alpha_t(i) \beta_t(i)}{\mathcal{L}_t} \quad (16.3.18)$$

Further, it follows from the definitions (16.3.13) and (16.3.15) that $\mathcal{L}_t$ is actually independent of t , so we can omit the subscript t and calculate it only once. (In practice, one often renormalizes at each time t for greater numerical stability, however.) The value of $\mathcal{L}$ can be interpreted as the probability (or likelihood) of the whole data set, given the parameters of the model.

Equations (16.3.14) and (16.3.16), taken together, are called the *forward-backward algorithm* for state estimation in a hidden Markov model.

Translating HMM into code, we start with a structure that will hold the various quantities that come into play, and its constructor. You construct an HMM structure by specifying a transition probability matrix $\mathbf{A}$ (N.B.: not $\mathbf{A}^T$), a symbol probability matrix $b_{ik} \equiv b_i(k)$, and a vector of observed data $\mathbf{y}$.

```

hmm.h  struct HMM {
        Structure for a hidden Markov model and its methods.
        MatDoub a, b;           Transition matrix and symbol probability matrix.
        VecInt obs;             Observed data.
        Int fbdone;
        Int mstat, nob, ksym;    Number of states, observations, and symbols.
        Int lnrnm;
        MatDoub alpha, beta, pstate;  Matrices  $\alpha$ ,  $\beta$ , and  $P_i(t)$ .
        VecInt arnm, brnm;
        Doub BIG, BIGI, lhood;
        HMM(MatDoub_I &aa, MatDoub_I &bb, VecInt_I &obs);  Constructor; see below.
        void forwardbackward();  HMM state estimation.
        void baumwelch();        HMM parameter re-estimation.
        Doub loglikelihood() {return log(lhood)+lnrm*log(BIGI);}
        Returns the log-likelihood computed by forwardbackward().
};

HMM::HMM(MatDoub_I &aa, MatDoub_I &bb, VecInt_I &obss) :
    a(aa), b(bb), obs(obss), fbdone(0),
    mstat(a.nrows()), nob(obs.size()), ksym(b.ncols()),
    alpha(nob,mstat), beta(nob,mstat), pstate(nob,mstat),
    arnm(nob), brnm(nob), BIG(1.e20), BIGI(1./BIG) {
    Constructor. Input are the transition matrix aa, the symbol probability matrix bb, and the
    observed vector of symbols obss. Local copies are made, so the input quantities need not
    be preserved by the calling program.
    Int i,j,k;
    Doub sum;
    Although space constraints make us generally stingy about printing code for checking input,
    we will save you a lot of grief by doing so in this case. If you get "matrix not normalized"
    errors, you probably have your matrix transposed. Note that normalization errors <1% are
    silently fixed.
    if (a.ncols() != mstat) throw("transition matrix not square");
    if (b.nrows() != mstat) throw("symbol prob matrix wrong size");
    for (i=0; i<nob; i++) {
        if (obs[i] < 0 || obs[i] >= ksym) throw("bad data in obs");
    }
    for (i=0; i<mstat; i++) {
        sum = 0.;
        for (j=0; j<mstat; j++) sum += a[i][j];
        if (abs(sum - 1.) > 0.01) throw("transition matrix not normalized");
        for (j=0; j<mstat; j++) a[i][j] /= sum;
    }
    for (i=0; i<mstat; i++) {
        sum = 0.;
        for (k=0; k<ksym; k++) sum += b[i][k];
        if (abs(sum - 1.) > 0.01) throw("symbol prob matrix not normalized");
        for (k=0; k<ksym; k++) b[i][k] /= sum;
    }
}
}

```

Now, to actually do the forward-backward estimation, you call the function `forwardbackward`. This fills the matrix `pstate`, so that $pstate_{ti} = P_t(i)$. It also sets the internal variables `lhood` and `lnrm` so that the function `loglikelihood` returns the logarithm of $\mathcal{L}$. Don't be surprised at how large in magnitude this (negative) number can be. The probability of any *particular* data set of more than trivial length is astronomically small!

In the following code, the quantities `BIG`, `BIGI`, `arnm`, `brnm`, and `lnrm` all relate to dealing with values that would far underflow an ordinary floating format. The basic idea is to renormalize as necessary, keeping track of the accumulated number of renormalizations. At the end, when an α , a β , and an $\mathcal{L}$ are combined, probability values of reasonable magnitude result.

hmm.h

```

void HMM::forwardbackward() {
HMM forward-backward algorithm. Using the stored a, b, and obs matrices, the matrices alpha,
beta, and pstate are calculated. The latter is the state estimation of the model, given the data.
    Int i,j,t;
    Doub sum,asum,bsum;
    for (i=0; i<mstat; i++) alpha[0][i] = b[i][obs[0]];
    arnrm[0] = 0;
    for (t=1; t<nobs; t++) {
        Forward pass.
        asum = 0;
        for (j=0; j<mstat; j++) {
            sum = 0.;
            for (i=0; i<mstat; i++) sum += alpha[t-1][i]*a[i][j]*b[j][obs[t]];
            alpha[t][j] = sum;
            asum += sum;
        }
        arnrm[t] = arnrm[t-1];
        if (asum < BIGI) {
            ++arnrm[t];
            for (j=0; j<mstat; j++) alpha[t][j] *= BIG;
        }
    }
    for (i=0; i<mstat; i++) beta[nobs-1][i] = 1.;
    brnrm[nobs-1] = 0;
    for (t=nobs-2; t>=0; t--) {
        Backward pass.
        bsum = 0.;
        for (i=0; i<mstat; i++) {
            sum = 0.;
            for (j=0; j<mstat; j++) sum += a[i][j]*b[j][obs[t+1]]*beta[t+1][j];
            beta[t][i] = sum;
            bsum += sum;
        }
        brnrm[t] = brnrm[t+1];
        if (bsum < BIGI) {
            ++brnrm[t];
            for (j=0; j<mstat; j++) beta[t][j] *= BIG;
        }
    }
    lhood = 0.;
    for (i=0; i<mstat; i++) lhood += alpha[0][i]*beta[0][i];
    lnrnm = arnrm[0] + brnrm[0];
    while (lhood < BIGI) {lhood *= BIG; lnrnm++;}
    for (t=0; t<nobs; t++) {
        Get state probabilities from  $\alpha$ 's and  $\beta$ 's.
        sum = 0.;
        for (i=0; i<mstat; i++) sum += (pstate[t][i] = alpha[t][i]*beta[t][i]);
        The next line is an equivalent calculation of sum. But we'd rather have the normalization
        of the  $P_i(t)$ 's be more immune to roundoff error. Hence we do the above sum for
        each value of t.
        // sum = lhood*pow(BIGI, lnrnm - arnrm[t] - brnrm[t]);
        for (i=0; i<mstat; i++) pstate[t][i] /= sum;
    }
    fbdone = 1;
    Flag prevents misuse of baumwelch(), later.
}

```

You may be wondering how well forwardbackward is able to do at predicting the hidden states of Teen Life, given just a long string of output symbols. If we take the prediction to be the state with the highest probability at each time, then this is correct about 78% of the time. Another 17% of the time, the correct state has the second-highest probability, often when the top two probabilities are nearly equal. It is an important property of HMMs that the output is not only a prediction, but also a quantitative assessment of how “sure” the model is of that prediction.

16.3.2 Some Variations on HMM

HMM state estimation with the forward-backward algorithm is a very flexible formalism, and many variants are possible. For example, in decoding codes on a trellis, as we did above, the symbols 0 or 1 are emitted not by the states, but by the transitions between the states. If we want to use HMM for that problem (we will say more about this below), we must replace $b_i(k)$ with $b_{ij}(k)$, the probability of emitting symbol k in a transition from state i to state j . The forward and backward recurrences now become

$$\begin{aligned}\alpha_{t+1}(j) &= \sum_{i=0}^{M-1} \alpha_t(i) A_{ij} b_{ij}(y_{t+1}) \\ \beta_{t-1}(i) &= \sum_{j=0}^{M-1} A_{ij} b_{ij}(y_t) \beta_t(j)\end{aligned}\tag{16.3.19}$$

and we start off the α 's with the special rule $\alpha_0(i) = 1$, since (like the case of $\beta_{N-1}(i)$ previously) the probability of the data is 1 before there are any data.

Another variant case is where one or more intermediate states are known exactly. In that case, one or more of the sums over $i_0, i_1, \dots, i_{t-1}$ in equation (16.3.13) is left out, and the corresponding index on an A and b gets replaced by the known state number. If you track through how this affects the recurrence equation (16.3.14), you'll see that the new procedure is

- calculate the α 's forward to, and including, a known state;
- zero all the α values at that time except for that of the known state;
- don't renormalize anything (though you feel tempted to do so); and
- continue forward with the α 's for the next timestep.

Proceed similarly for the β 's.

The opposite variant is where you have *missing data*, meaning that for some values of t there is no observation of the symbol y_t . In this case, all you need to do is to make a special case for the symbol probability,

$$b_i(y_t) \equiv 1, \quad (0 \leq i < M) \quad t \in \{\text{missing}\}\tag{16.3.20}$$

meaning that, regardless of state i , the probability of observing the data (meaning no data) at time t is unity. Now proceed as usual to calculate the state probabilities. If you then want to *reconstruct* the missing data, you can calculate its posterior probabilities,

$$P(y_t = k \mid \mathbf{y}) = \sum_{i=0}^{M-1} P_i(t) b_i(k) = \sum_{i=0}^{M-1} \frac{\alpha_i(t) \beta_i(t)}{\mathcal{L}} b_i(k) \quad t \in \{\text{missing}\}\tag{16.3.21}$$

16.3.3 Bayesian Re-Estimation of the Model Parameters

This is magical. The probability that we were in state i at time t is $\alpha_t(i) \beta_t(i) / \mathcal{L}$. What is the probability, given the data $\mathbf{y}$, that a given transition, say between time

t and time $t + 1$, was a transition between state i and state j ? We write, applying various of the laws of probability,

$$\begin{aligned}
 P(s_t = i, s_{t+1} = j \mid \mathbf{y}) &= P(s_{t+1} = j \mid \mathbf{y}, s_t = i) P(s_t = i \mid \mathbf{y}) \\
 &= \frac{P(\mathbf{y} \mid s_{t+1} = j, s_t = i) P(s_{t+1} = j \mid s_t = i)}{\sum_j P(\mathbf{y} \mid s_{t+1} = j, s_t = i) P(s_{t+1} = j \mid s_t = i)} P(s_t = i \mid \mathbf{y}) \\
 &= \frac{[\alpha_t(i) b_j(y_{t+1}) \beta_{t+1}(j)] [A_{ij}]}{\sum_j [\alpha_t(i) b_j(y_{t+1}) \beta_{t+1}(j)] [A_{ij}]} \frac{[\alpha_t(i) \beta_t(i)]}{\mathcal{L}} \\
 &= \frac{\alpha_t(i) A_{ij} b_j(y_{t+1}) \beta_{t+1}(j)}{\mathcal{L}} \tag{16.3.22}
 \end{aligned}$$

Note how the sum over j in the denominator disappears by the recurrence (16.3.16) for $\beta_t(i)$.

So, for a long run of data, we can compute the fraction of the time that a state i transitions to state j as the estimated number of $i \rightarrow j$ transitions divided by the estimated number of i states,

$$\hat{A}_{ij} = \frac{\sum_t \alpha_t(i) A_{ij} b_j(y_{t+1}) \beta_{t+1}(j)}{\sum_t \alpha_t(i) \beta_t(i)} \tag{16.3.23}$$

noting that the $\mathcal{L}$'s cancel out. The reason for calling this quotient $\hat{A}_{ij}$ is that it is a *re-estimation* of the transition probability A_{ij} . The corresponding re-estimation of the symbol probability matrix $b_i(k)$ is the fraction of all i states that emit a symbol k , namely

$$\hat{b}_i(k) = \frac{\sum_t \delta(y_t, k) \alpha_t(i) \beta_t(i)}{\sum_t \alpha_t(i) \beta_t(i)} \tag{16.3.24}$$

where $\delta(j, k)$ is 1 if $j = k$, zero otherwise.

You might think that this process is somehow circular, or that re-estimating A_{ij} and $b_i(k)$ in this fashion only introduces noise that degrades the model. Far from it! Baum and Welch first showed that replacing A_{ij} by $\hat{A}_{ij}$ and $b_j(k)$ by $\hat{b}_j(k)$, and then recalculating the probabilities of each state at each time by the forward-backward algorithm, always *increases* $\mathcal{L}$, the overall likelihood of the model. It is, in fact, an EM algorithm (cf. §16.1, and see below). You can continue this cycle of estimating states (forward-backward) and re-estimating model probabilities (Baum-Welch), obtaining further increases in $\mathcal{L}$, until convergence to a maximum is achieved. Equations (16.3.23) and (16.3.24) are known as *Baum-Welch re-estimation*.

So the magic is this: We began by estimating states in a *known* hidden Markov model. We now see that, *just from the data*, we can get not only an estimate of the states, but also an estimate of the model itself, that is, the transition probabilities and symbol probabilities. Like any iterative process, this works best if we have a good initial guess. But it will often converge to a good model from a fairly random initial guess. (You should not start with exactly uniform probabilities, because that creates a symmetry that the iteration finds hard to break.)

The code is straightforward. The updating of $b_i(k)$ comes almost for free as a byproduct of computing the denominator in the update for A_{ij} . Like the forward-backward algorithm, Baum-Welch re-estimation takes $O(NM^2)$ operations.

```

hmm.h void HMM::baumwelch() {
    Baum-Welch re-estimation of the stored matrices a and b, using the data obs and the matrices
    alpha and beta as computed by forwardbackward() (which must be called first). The previous
    values of a and b are overwritten.
    Int i,j,k,t;
    Doub num,denom,term;
    MatDoub bnew(mstat,ksym);
    Doub powtab[10];           Fill table of powers of BIGI.
    for (i=0; i<10; i++) powtab[i] = pow(BIGI,i-6);
    if (fbdone != 1) throw("must do forwardbackward first");
    for (i=0; i<mstat; i++) {   Looping over i, get denominators and new b.
        denom = 0.;
        for (k=0; k<ksym; k++) bnew[i][k] = 0.;
        for (t=0; t<nobs-1; t++) {
            term = (alpha[t][i]*beta[t][i]/lhood
                    * powtab[arnrm[t] + brnrm[t] - lnrnm + 6];
            denom += term;
            bnew[i][obs[t]] += term;
        }
        for (j=0; j<mstat; j++) { Inner loop over j gets elements of a.
            num = 0.;
            for (t=0; t<nobs-1; t++) {
                num += alpha[t][i]*b[j][obs[t+1]]*beta[t+1][j]
                       * powtab[arnrm[t] + brnrm[t+1] - lnrnm + 6]/lhood;
            }
            a[i][j] *= (num/denom);
        }
        for (k=0; k<ksym; k++) bnew[i][k] /= denom;
    }
    b = bnew;
    fbdone = 0;
}
    Don't let this routine be called again until forward-
    backward() has been called.

```

You must always precede a call to `baumwelch` by a call to `forwardbackward`, since the latter updates the α and β tables. Also, as you alternate calls to the two functions, you monitor convergence by the value of the log-likelihood calculated by `forwardbackward`.

Be aware that convergence can be excruciatingly slow! The references describe methods by which convergence can be accelerated in some cases. Common difficulties are when a rare state is not correctly captured by the model, or when the model thinks that there are two states, with nearly identical transition probabilities, when there is really only one. If you have even a plausible guess for the transition probability matrix, you should use it to start. There are many applications where you shouldn't use re-estimation at all: If you have a pretty good model to start with, just use it (via `forwardbackward`) to analyze your data, and don't even think about re-estimating.

The Baum-Welch re-estimation algorithm, which dates from the mid-1960s, was generalized in the mid-1970s by Dempster, Laird, and Rubin, as the *expectation-maximization (EM) algorithm*, with a variety of applications to problems with missing or censored data. (An example is the Gaussian mixture model in §16.1.) In this more general language, the forward-backward algorithm is an E-step, while Baum-Welch is an M-step. Alas, one small cloud in an otherwise bright sky is that the maximum of $\mathcal{L}$ achieved by multiple EM iterations is only guaranteed to be a local, not a global, maximum.

HMM has found wide application in speech recognition, gene sequence comparison, financial models, and a variety of other fields. The references give specifics.

16.3.4 Comparing the Viterbi Algorithm with HMM

It is important to understand the similarities and differences between the Viterbi algorithm and hidden Markov modeling (its forward-backward algorithm in particular).

When we discussed the Viterbi algorithm in the context of decoding, we made the implicit assumption that a 1 bit was a priori as likely as a 0 bit. It is straightforward to generalize the Viterbi algorithm to include an arbitrary a priori transition probability A_{ij} , just like HMM. In that case, the probability factor on each edge (whose negative logarithm is the edge cost) is the product of two terms, again just like HMM, $A_{ij}b_{ij}(k)$, where now $b_{ij}(k)$ is the probability of observing the observed symbol k given that an $i \rightarrow j$ transition occurred.

We discussed Baum-Welch parameter re-estimation for HMMs in some detail. Re-estimation of the parameters in a Viterbi model, often called *Viterbi training*, is analogous. Take the most probable path output by the algorithm (or ensemble of paths collected from the decodes of many codewords). Count the number of $i \rightarrow j$ transitions seen along these paths and the numbers of each symbol k seen for each pair i, j . Now re-estimate A_{ij} and $b_{ij}(k)$ by the obvious normalizations of these counts.

The Viterbi algorithm and the forward estimation part of the forward-backward algorithm are structurally very similar. In both cases, we sweep forward in time (or by stages) and assign a likelihood (or posterior probability) to each node, based on the data already seen. The difference is that Viterbi assigns to a node the probability of the *single best path* that reaches it, while forward-backward assigns the sum of probabilities over *all possible paths* to that node. Indeed the Viterbi algorithm is sometimes called the *min-sum algorithm* while forward-backward is referred to as the *sum-product algorithm*, just to highlight this distinction. (In the context of coding theory, the forward-backward algorithm is also sometimes called the *BCJR* or *Bahl-Cocke-Jelinek-Raviv* algorithm. In other contexts it is sometimes called *belief propagation*.)

The backward passes of the two algorithms have somewhat different structures. For Viterbi, the backward pass simply consolidates the information about the single most probable path that is already implicit in the node labeling. For forward-backward, as we have seen, the backward pass is needed to get posterior probabilities for each node that use all the data, both ahead of and behind any time t .

If you think you have a choice between using the Viterbi algorithm or using HMM, you should probably think again. Most problems clearly favor only one or the other method. If your desired output is a valid *path* on the graph, then HMM won't do: It might yield a set of highly probable nodes that just don't lie on any single path. For example, you might have the first half of one codeword and the second half of another, with no graph edge connecting the two halves. That is why decoding theory usually starts in the world of Viterbi (although, in some more complicated constructs, it can end up with a foot in each world).

On the other hand, if you care about which *nodes* are visited, then HMM is most likely what you want. In fact, Viterbi can give very poor results. The most probable path is often *very improbable* when compared to the sum of all paths that lead through a particular node, one possibly not on the most probable path. Or, another way of describing this, there may be exponentially many paths with not-too-different probabilities, so the node probabilities are determined by the statistics of where they